

From *Sumy* to Simplicity: Revitalizing Extractive Summarization With Improved Speed and Accuracy

1st Osiris J. Terry

Department of Electrical and Computer Engineering
Air Force Institute of Technology
2950 Hobson Way, Wright-Patterson AFB, OH 45433
osiris.terry.4@au.af.edu

2nd Christine M. Schubert Kabban PhD

Department of Mathematics and Statistics
Air Force Institute of Technology
2950 Hobson Way, Wright-Patterson AFB, OH 45433

3rd Kenneth M. Hopkinson PhD

Department of Electrical and Computer Engineering
Air Force Institute of Technology
2950 Hobson Way, Wright-Patterson AFB, OH 45433

Abstract—Recent advances in deep learning have shifted the focus of Natural Language Processing (NLP) towards transformer-based models, which are especially effective for abstractive summarization. While these models offer superior semantic understanding, they come with significant challenges, including high computational costs, large training data requirements, and difficulties in adapting to new or evolving concepts. In contrast, traditional Bag-of-Words (BoW) models, despite being overshadowed by newer techniques, have regained attention for tasks like extractive summarization, particularly when integrated into retrieval-augmented generation (RAG) frameworks. These BoW methods, such as BM25, offer efficiency in document retrieval, while dense models handle more complex semantic tasks. Despite the growing demand for efficient BoW-based summarization tools, existing solutions like *Sumy* lack comprehensive documentation and customization options. This paper addresses this gap by re-implementing key extractive summarization algorithms in Python, using the Natural Language Toolkit (NLTK) for preprocessing and NumPy for computation. We compare our implementation with *Sumy* and demonstrate significant improvements, including a 19.43% increase in Rouge-N scores and a performance boost of up to 101.5x faster for larger token sets. Our work contributes toward revitalizing these techniques by offering a practical, extensible alternative that enables researchers and developers to move beyond black-box usage and toward a deeper, more accurate understanding of extractive summarization algorithms.

Index Terms—Text Summarization, Bag-of-Words (BoW), Retrieval Augmented Generation (RAG), Computational Efficiency, *Sumy*

I. INTRODUCTION

Bag-of-Words (BoW) models have long been a foundational technique in Natural Language Processing (NLP), widely used in tasks such as document clustering, summarization, question answering, and topic modeling. Their effectiveness lies in a straightforward yet powerful approach that leverages statistical word frequency to represent text. However, with the advent of more advanced machine learning techniques, particularly deep learning models, BoW models began to lose favor. Neural network and transformer-based models, especially those used for

abstractive summarization, shifted the research community’s focus due to their ability to generate human-like summaries by understanding the context and meaning of text. Unlike the BoW approach, which primarily extracts information based on word frequency, these newer models generate summaries that reflect deeper semantic understanding.

While these deep learning models are highly effective in producing detailed abstractive summaries, they come with significant challenges. They require vast amounts of training data and substantial computational resources, making them less accessible to many users. Additionally, they are prone to overfitting and struggle in generalizing to unseen data. One of the most significant limitations of these dense models is their inability to adapt to new words or concepts that were not present in their training data, which can be problematic in rapidly evolving domains.

In recent years, however, there has been a resurgence of interest in traditional statistical techniques, such as BoW models, due to their complementary strengths in tasks like extractive summarization. This renewed interest can be largely attributed to the rise of retrieval-augmented generation (RAG), where BoW-based models are used in tandem with dense models. In this hybrid approach, BoW techniques like TextRank are employed for efficient document summarization, while dense models take over the more complex task of semantic understanding. This combination leverages the adaptability of BoW models and the context-awareness of deep learning models, resulting in stronger performance in applications like information retrieval, document ranking, and summarization [1].

Amidst the renewed interest in classical NLP approaches, there is growing demand for modern tools capable of efficiently implementing BoW-based summarization methods. While many contemporary NLP libraries prioritize advanced deep learning techniques, there remains a noticeable gap in support for streamlined, well-documented BoW-based summarization solutions. One of the most widely used libraries in

this space is *Sumy*, valued for its simplicity and speed, which has made it a common choice in both academic and practical settings. *Sumy* supports a range of generic extractive summarization algorithms, including Luhn, SumBasic, Kullback-Leibler (KL), Latent Semantic Analysis (LSA), Graph Reduction, TextRank and LexRank.

Despite its utility, *Sumy* poses several limitations—most notably its sparse documentation and the lack of guidance for optimizing and customizing these algorithms. These challenges largely stem from the tool’s origin: it was originally developed in Czech/Slovak as part of a diploma thesis. As the author notes:

“Sumy was created as my diploma thesis and the need for articles length reduction in Czech/Slovak language. Although its source code was always available publicly on GitHub I didn’t expect to adopt it by so many people. Don’t get me wrong. I am happy for it, but that’s also why the lack of documentation and sometimes hardcoded features for Slovak/Czech languages may be found in the codebase.” [2]

This context helps explain some of the design decisions and limitations, and underscores the need for a more robust, transparent, and adaptable alternative for researchers and developers working with extractive summarization. Furthermore, many users are familiar with Gensim’s summarization algorithm which implemented TextRank, but they have since removed it due to performance issues. This leaves *Sumy* as the most prominent and diverse package for extractive text summarization.

This paper is not intended as a critique of the authors of *Sumy*, but rather as a critical examination of the broader practice of relying on prebuilt NLP packages without fully understanding or being able to control their individual steps, after which we offer a practical solution. Specifically, this work equips researchers with the tools and insights needed to adapt foundational extractive summarization algorithms to their specific use cases. By decoupling these algorithms from opaque preprocessing pipelines, we enable more accurate assessments of their performance and potential improvements. To achieve this, we:

- 1) Review existing literature that uses *Sumy* as a benchmark or solution.
- 2) Re-implement core extractive summarization algorithms using only Natural Language Toolkit (NLTK) for preprocessing and NumPy for computation.
- 3) Compare our implementation results against those produced by *Sumy*
- 4) Summarize our contributions and propose future directions.

COMPREHENSIVE REVIEW OF PACKAGE USAGE

We begin by examining how summarization packages like *Sumy* are currently used across the literature. A close review reveals three distinct patterns in usage: as baseline models

for comparison, as components in development pipelines, and as subjects of comparative evaluation studies. Each use case illustrates different assumptions and potential limitations of relying on packaged summarization tools, particularly when their internal logic is not transparent or customizable.

A. Packages as Baseline Methods

A common use of packages like *Sumy* is to serve as baseline methods against which new models are evaluated. For example, one study proposed an unsupervised extractive multi-document summarization method using BERT sentence embeddings and demonstrated its superiority by comparing it to several baseline methods, including those implemented in *Sumy*. The results showed that the new method significantly outperformed the baselines, especially after fine-tuning BERT with multi-task learning [10]. While these comparisons are informative, they rely on the assumption that the package’s implementation is both accurate and consistent across different datasets. Without visibility into the internal workings of the package, these assumptions can be misleading and may exaggerate the reported performance of the proposed methods.

B. Packages in Development Pipelines

Summarization packages are also used within larger product pipelines, where they support practical tasks such as content reduction, summarization, or segmentation. In one example, a university-developed tool used crowd-sourced timestamps and automatic transcription to segment and summarize lecture videos, thereby reducing storage requirements and improving content accessibility. This pipeline relied on a summarization component to condense the transcribed text [16]. However, such reliance introduces potential bottlenecks—if the package underperforms or lacks customization options, the overall product suffers. Tailoring individual components of the pipeline, especially preprocessing and summarization steps, could significantly enhance performance, but is often infeasible when working with black-box packages.

C. Package Comparison Studies

A smaller set of studies focuses on direct package comparisons. One such study evaluated summaries produced by *Sumy* and a Term-Frequency Inverse Document Frequency (TF-IDF) based method built with NLTK, finding that the latter outperformed *Sumy* by nearly 40% on BLEU scores [20]. Such stark differences raise important questions about the quality and consistency of packaged implementations. These results also highlight a key challenge: it is often unclear whether performance differences stem from algorithmic design, implementation quirks, mismatched preprocessing, or specific test data sources. The lack of transparency in many packages prevents meaningful diagnosis and reduces confidence in their comparative utility.

D. Limitations and Implications

Taken together, these use cases demonstrate that while summarization packages provide convenience and accessibility,

they often obscure the very processes that are critical for rigorous evaluation and meaningful adaptation. Whether used as benchmarks, embedded in pipelines, or compared in studies, their black-box nature limits interpretability, reproducibility, and customizability.

This motivates our core contribution: a clear and simple implementation of common extractive summarization algorithms with fully separated preprocessing and computation components. By removing the opacity and rigidity of prebuilt packages, we aim to offer a clearer foundation for evaluation and adaptation—one that researchers and developers can trust, extend, and tailor to their specific needs. While we highlight only a few representative examples in the discussion above, table I presents a broader selection of papers, each categorized according to one of the three identified themes.

Theme	References
Used <i>Sumy</i> as a baseline for comparison against other models	[3], [4], [5], [6], [7], [8], [9], [10], [11], [12]
Used <i>Sumy</i> as part of a product pipeline	[13], [14], [15], [16], [17], [18]
Used <i>Sumy</i> within a package comparison study	[19], [20], [23], [24]

TABLE I

Sumy REVIEW: SUMMARY OF THEMES

II. MATRIX IMPLEMENTATION OF SUMMARIZATION TECHNIQUES

In this section, we present our versions of Luhn, SumBasic, KL, LSA, Graph Reduction, and TextRank. All of which generate summaries solely through matrix mathematics by leveraging a latent term frequency matrix created during the preprocessing stage.

A. Preprocessing

The preprocessing steps described here are consistently applied across all methods. In implementing these methods, several key considerations must be addressed to optimize performance. The methods must be fast, accurate, and scalable, which requires minimizing computational complexity, particularly in terms of $O(N)$ notation.

To generate the input matrix, we need to process the text token by token. Fortunately, efficient libraries, such as NLTK, provide robust tools for this task. We will use NLTK’s word tokenizer to break the input text into tokens, and incorporate token stemming to better capture the frequency of each term (e.g., converting “going” to “go”). Specifically, we will apply the snowball stemming algorithm for this purpose. During token iteration, we will also filter out stop words—terms that hold little semantic value. Finally, after processing all tokens, we will convert the resulting data structure into a matrix, where each row represents a latent term (meaning it will no longer be referenced explicitly, but instead as a row) and each column represents a sentence of the text. An example is shown in Figure 1 and Figure 2.

Terms		Sentences				
		S1	S2	S3	S(N)	
		Hat	1	0	0	...
		Road	0	2	0	...
		Signifi	0	1	1	...
		Truck	1	2	1	...
		...	...	...	...	...

Fig. 1. Term Frequency Matrix

	Sentences			
	1	0	0	...
	0	2	0	...
	0	1	1	...
	1	2	1	...
	...	...	...	...

Fig. 2. Latent Term Frequency Matrix

In Figure 1, observe that the stem “signifi” could correspond to words such as “significantly” or “significant”. Also, note the absence of stop words (e.g., “the,” “a,” “an,” “in”), and how each value in the matrix represents the frequency of term X (row index) within sentence Y (column index). In Figure 2, we transform the data structure into a matrix where the terms are represented by the T rows, while the columns correspond to N sentences. This transformation reflects the fact that, after preprocessing, we should not need to revisit the text to generate a summary. Instead, we can rely entirely on the information captured during the preprocessing phase. In the following section, this latent matrix will be referred to as the **TF** matrix. Lastly, we implemented the preprocessing step in the following code using NLTK and NumPy:

```

1 def preprocess(text: Union[str, List[str]]) ->
  np.ndarray:
2     if isinstance(text, str):
3         text = splitter.sentences(text)
4         text = tokenize(text)
5     elif isinstance(text, list):
6         text = tokenize(text)
7
8     num_sentences = len(text)
9     term_dict = {}
10
11     for i, sentence in enumerate(text):
12         for token in sentence:
13             token = token.lower()
14             if not token in stop_words and not
15                 token in string.punctuation and
16                 token.strip() != '':
17                 base_token = stem(token)
18                 if base_token not in term_dict:

```

```

17         term_dict[base_token] =
18             np.zeros(num_sentences)
19         term_dict[base_token][i] += 1
20
21     tf_matrix =
22         np.vstack(list(term_dict.values()))
23     return tf_matrix

```

```

4         scores = (TF *
5                     TF.sum(axis=1).reshape(-1,
6                     1)).sum(axis=0)
7
8         # Return the top 3 indexes of the
9         # highest scores
10        return [text[int(idx)] for idx in
11                np.argsort(scores)[::-1][:top_k]]

```

B. Luhn Summarization

Luhn’s method for automatic summarization involves several key steps that focus on the frequency of words within a document and the relevance of sentences. Luhn proposed that words that appear frequently in a document are likely to carry significant meaning [24]. We implemented Luhn’s method using our latent term matrix generated from our preprocessing step:

- 1) **Summing across rows of the TF matrix:** This generates a column vector, $\mathbf{v}$, where each element represents the frequency of a term across the total number of sentences, N .

$$\mathbf{v}_i = \sum_{j=1}^N \mathbf{TF}_{i,j} \quad \text{for each } i = 1, 2, \dots, T \quad (1)$$

- 2) **Multiplying by the $\mathbf{v}$ vector:** Each element of the $\mathbf{v}$ column vector is multiplied element-wise across each row of the $\mathbf{TF}$ matrix. This operation treats the $\mathbf{TF}$ matrix as a unit vector and uses the $\mathbf{v}$ column vector to generate the magnitude.

$$\mathbf{M} = \mathbf{v} \circ \mathbf{TF} \quad (2)$$

- 3) **Calculating sentence scores:** For each sentence, we compute a score by summing the weights of all terms associated with that sentence. This produces a vector of sentence-level scores, which we refer to as the **scores** vector.

$$\text{scores}_j = \sum_{i=1}^T M_{i,j} \quad \text{for each } j = 1, 2, \dots, N \quad (3)$$

We can easily implement the same logic using NumPy, a high-performance library for numerical computing in Python. Its core functionality, written in C, ensures efficient computations, especially with large datasets. NumPy also uses broadcasting for matrix operations—e.g., multiplying a vector $\mathbf{v}$, $(T \times 1)$, with a matrix $\mathbf{TF}$, $(T \times N)$, by broadcasting the vector into a $(T \times N)$ matrix before performing the multiplication.

```

1     def luhn(text: Union[str, List[str]], top_k:
2         int = 3 ):
3         TF = preprocess(text)
4         # 1. Multiply the TF by the sum of the
5         # rows, creating frequency word
6         # scores for each sentence, then sum
7         # the columns to get the sentence
8         # scores.

```

C. SumBasic

SumBasic is a more recent summarization technique that shares conceptual similarities with the Luhn summarization method discussed earlier. Both techniques are grounded in the fundamental idea of using word frequency to generate summaries [25]. However, what sets SumBasic apart from Luhn is its focus on multi-document summarization, which aims to minimize redundancy when creating summaries. In multi-document summarization, redundancy is often inevitable as multiple documents may contain overlapping information. Therefore, SumBasic is designed to account for this redundancy by selectively prioritizing unique information. SumBasic assigns importance to words based on their probability of appearing in the text. This approach emphasizes that high-frequency words are more likely to appear in a human-generated summary, same as Luhn. Each sentence is scored based on the average word probability and the top sentence chosen. Typically, we will need multiple sentences, but we need to adjust the sentence scores based on the sentence that was chosen. Thus, the probabilities of all the words within the chosen sentence are squared, penalizing sentences that contain similar words. Lastly, the steps above are repeated until we have reached the desired summary length, set by the user. Using only the TF matrix we can implement SumBasic:

- 1) **Create Binary Matrix $\mathbf{A}$:** Generate a matrix of the same size as the TF matrix that contains a 1 or 0 depending on if term i is within sentence j .

$$\mathbf{A}_{ij} = \begin{cases} 1 & \text{if } TF_{ij} \neq 0 \\ 0 & \text{if } TF_{ij} = 0 \end{cases} \quad (4)$$

- 2) **Create Global Probability Vector:** Generate a vector $\mathbf{p}$ where each element is the probability of a term within all of the text.

$$\mathbf{p}_i = \frac{\sum_{j=1}^N \mathbf{TF}_{i,j}}{T} \quad \text{for each } i = 1, 2, \dots, T \quad (5)$$

- 3) **Create Sentence Score Vector:** Generate a vector where each element is the score of the respective sentence. To reiterate, the score is the average of the global word probabilities within that sentence.

$$\text{scores}_j = \frac{\mathbf{p} \circ \mathbf{A}}{\sum_{i=1}^T \mathbf{A}_{i,j}} \quad \text{for each } j = 1, 2, \dots, N \quad (6)$$

- 4) **Select Top Sentence:** Choose the highest scoring sentence k .

- 5) **Update Probabilities:** Square the probabilities of the words within the chosen sentence k . For efficiency, we will do this using the matrices we previously generated.

$$\mathbf{a}_i = A_{i,k} \quad \text{for each } i = 1, 2, \dots, T \quad (7)$$

$$\mathbf{p}_i^{\text{new}} = \mathbf{p}_i \circ (\mathbf{p}_i \circ \mathbf{a}_i + \mathbf{1} - \mathbf{a}_i) \quad (8)$$

First, we create a mask $\mathbf{a}_i$ via the binary vector of the chosen sentence k . Next, we multiply this mask with the probability vector $\mathbf{p}_i$. This generates a vector that contains only the probabilities of the latent terms that are currently selected in the sentence. We then add 1 to this vector and subtract the mask $\mathbf{a}_i$, which replaces the 0's with 1's for the non-selected latent terms within the vectors. Finally, we multiply the vector by the original probability vector $\mathbf{B}$ to obtain the updated probability vector. This final step has the effect of squaring the probabilities for the terms that have been selected, thus reducing their likelihood of being selected again in future iterations.

- 6) **Repeat:** Repeat steps 3-5 until the desired summary length has been reached.

Using NumPy we implemented SumBasic with the following code:

```

1  def SumBasic(text: Union[str, List[str]],
2      top_k: int = 3 ):
3      TF = preprocess(text)
4      results = []
5      # 1. Create global word probabilities
6      p = TF.sum(axis=1)/TF.sum()
7      A = (TF > 0).astype(int)
8      while len(results) < top_k:
9          # 2. Dot p by the TF matrix and divide
10             by the total number of words in the
11             sentence to generate the sentence scores
12          scores = (np.dot(TF.T, p.reshape(-1,
13              1))/np.sum(A, axis=0)).flatten()
14          # 3. Choose the top unique sentence
15          top_idx = next((index for index in
16              np.argsort(scores)[-1] if index
17              not in results), None)
18          results.append(top_idx)
19          # 4. Update the p vector
20          p = p * (p*A[:, top_idx] + 1 - A[:,
21              top_idx])
22
23      return [text[int(idx)] for idx in results
24          if idx is not None]
```

D. KL Summarization

The Kullback-Leibler (KL) divergence summarization is conceptually similar to SumBasic in that it calculates word probabilities and accounts for redundancy, making it suitable for in multi-document summarization [26]. KL divergence quantifies how much one probability distribution diverges from a reference distribution. In the context of summarization, KL divergence measures the "distance" between the word distributions of individual sentences and the overall document. This divergence is computed as:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right) - P(x) + Q(x) \quad (9)$$

Where $P(x)$ is the reference distribution and $Q(x)$ is the approximating distribution. Similar to SumBasic, sentences are selected iteratively. At each iteration, sentence scores are computed based on the similarity between a sentence's word distribution and the word distribution of the entire corpus. Once a sentence is selected, it is removed from further consideration. In subsequent iterations, sentence scores are recomputed to account for sentences that have already been selected. This process continues until the desired summary length is reached. Mathematically, the resulting summary represents the optimal combination of sentences whose combined word distribution is closest to that of the corpus.

We implemented KL Summarization using the TF matrix in the following steps:

- 1) **Initialize Rolling Summary Vector:** Generate a vector $\mathbf{S}$ that contains the word frequencies of the current summary.

$$\mathbf{s}_i = 0 \quad \text{for each } i = 1, 2, \dots, T \quad (10)$$

- 2) **Create Global Probability Vector:** Generate a vector where each element is the probability of a term within all of the text. This is the same as equation (5).

- 3) **Create Local Probability Matrix:** Generate a matrix where each element is the probability of term i appearing in the combination of the current summary with sentence j .

$$\text{colsum}(q)_j = \sum_{i=1}^T (\mathbf{TF}_{i,j} + \mathbf{s}_i) \quad \text{for each } j = 1, 2, \dots, N \quad (11)$$

We do this by creating a dummy vector $\text{colsum}(q)$ whose elements contain the total number of words within each sentence in the TF matrix combined with the current rolling summary sentences.

$$\mathbf{Q}_{i,j} = \frac{\mathbf{TF}_{i,j} + \mathbf{s}_i}{\text{colsum}(q)_j} \quad (12)$$

Then we element-wise add the column vector $\mathbf{S}$ to each column of the TF matrix. Lastly, we divide by $\text{colsum}(q)$ to generate the probabilities, shown in equation (12).

- 4) **Generate scores using KL divergence:** Create the sentence scores vector by calculating the KL divergence between the global word probabilities $\mathbf{p}_i$ and the columns of the $\mathbf{Q}$ matrix.

$$\text{scores}_j = \sum_{i=1}^T \mathbf{p}_i \cdot \log \left(\frac{\mathbf{p}_i}{\mathbf{Q}_{i,j}} \right) - \mathbf{p}_i + \mathbf{Q}_{i,j} \quad \text{for each } j = 1, 2, \dots, N \quad (13)$$

- 5) **Select Top Sentence:** Choose the lowest KL divergence as the top scoring sentence k and update the rolling summary vector.

$$\mathbf{s}_i = \mathbf{s}_i + \mathbf{Q}_{i,k} \quad \text{for each } i = 1, 2, \dots, T \quad (14)$$

- 6) **Repeat:** Repeat steps 3-5 until the desired summary length has been reached, which was set by the user.

Using NumPy, we implemented the KL summarization algorithm with the following code. It is important to note that we clip zero values to a small positive number during division to prevent division by zero, which would result in a "NaN" (Not a Number) in Python:

```

1  def KLSum(text: Union[str, List[str]], top_k:
2      int = 3 ):
3      TF = preprocess(text)
4      results = []
5      # 1. Calculate Word probabilities of
6      # document and summary vector
7      q = (TF.sum(axis=1)/TF.sum()).reshape(-1,
8          1)
9      s = np.zeros(TF.shape[0]).reshape(-1, 1)
10     while len(results) < top_k:
11         # 2. Create Q Matrix
12         Q = (TF + s)/np.clip(np.sum(TF + s,
13             axis=0), 1, None)
14         # 3. Calculate KL divergence for each
15         # column
16         scores = np.sum(np.log(B/np.clip(Q,
17             1e-100, 1.0)) * q - q + Q, axis=0)
18         # 4. Get the top idx of the lowest KL
19         # divergence
20         top_idx = next((index for index in
21             np.argsort(scores) if index not in
22             results), None)
23         results.append(top_idx)
24         # 5. Update the summary vector
25         s = s + (TF[:, top_idx]).reshape(-1, 1)
26
27     return [text[int(idx)] for idx in results
28         if idx is not None]
```

E. LSA Summarization

LSA is a powerful technique in NLP commonly applied to document clustering, topic modeling, and summarization. It leverages Singular Value Decomposition (SVD) on the input matrix. In this case, we will use the TF Matrix as it simplifies the computations [27]. To reiterate, the rows represent terms and columns represent sentences, with entries indicating the frequency of a term in a particular sentence. SVD decomposes a matrix A into three matrices: U , Σ , and V^T , expressed as:

$$A = U\Sigma V^T \quad (15)$$

Here, A is the original matrix, U contains the left singular vectors, Σ holds the singular values, and V^T contains the right singular vectors. The decomposition reveals latent topics and their contributions to terms and sentences. First, we compute the eigenvalues and eigenvectors of the original matrix. Eigenvectors represent latent topics, and eigenvalues reflect their importance. The Σ matrix organizes these values, indicating the strength of each latent topic within a diagonal matrix, while the matrices U and V describe the relations between terms, topics, and sentences. Specifically, each element of U is the contribution of term i to latent topic j and each element of V is the contribution of sentence i to latent topic j . By using

these matrices, we can compute the significance of terms and sentences within specific latent topics.

Using LSA to generate extractive summaries involves performing SVD on the TF matrix and ranking sentences by their contributions to latent topics based on each topics respective importance:

- 1) **Compute the SVD the TF Matrix:** Decompose the TF matrix using SVD.

- 2) **Compute the sentence scores:** The sentence scores are calculated by multiplying the absolute values of V with the singular values Σ through element-wise multiplication and summation across the columns:

$$\text{scores}_j = \sum_z \text{# of latent topics} |V_{i,z}| \cdot |\Sigma_{z,z}| \quad \text{for each } j = 1, 2, \dots, N \quad (16)$$

The sentences are ranked in descending order based on their scores, and the top sentences are returned.

Using NumPy, we implemented the LSA summarization algorithm with the following code. In NumPy, the Σ is a vector instead of a diagonal matrix where the latent topics close to zero are cut off, allowing us to truncate the V matrix.

```

1  def LSASum(text: Union[str, List[str]],
2      top_k: int = 3 ):
3      TF = preprocess(text)
4      # 1. Perform SVD on the TF
5      _U, s, Vt = np.linalg.svd(TF)
6      # 2. Truncate the Vt matrix
7      Vt = Vt[:len(s), :]
8      # 3. Calculate the sentence scores
9      sentence_scores = np.sum(np.abs(Vt) *
10         np.abs(s.reshape(-1, 1)), axis=0)
11      # 4. Return the top_k sentences
12
13     return [text[int(idx)] for idx in
14         np.argsort(sentence_scores)[::-1][:top_k]]
```

F. Graph Reduction

Graph reduction is one of the most straightforward and effective approaches in document summarization [2]. In graph-based summarization, sentences are represented as nodes in a graph, and the relationships between these sentences are represented as edges connecting the nodes. These graphs are typically undirected and can either be weighted or unweighted:

- **Unweighted graphs** imply that the relationships between sentences are treated equally, meaning the edges have the same strength or size.
- **Weighted graphs** imply that the edges have varying strengths, represented by edge weights, which reflect the strength or importance of the relationship between two sentences.

The relationships between sentences (or text segments) can be determined in various ways, such as:

- **Term overlap:** The number of common terms between sentences.
- **Cosine similarity:** A measure of similarity between the term-frequency vectors of sentences.

- **Text embeddings:** Calculating the distance between embeddings of sentences (this can combine BoW features with more sophisticated methods, resulting in a hybrid approach).

Using the concepts above, the graph reduction algorithm forms an adjacency matrix i.e. the matrix representation of a graph, then computes the sentence ranks by finding which sentences are the most connected within the graph. This method is already in matrix form and can be outlined in the following steps:

- 1) **Transform the TF matrix into a similarity matrix.** Calculate the cosine similarity between each sentence. This can be done by finding the dot product between the TF matrix and its transpose. Specifically, each element in the similarity matrix $SS_{i,j}$ is the dot product between sentence vector i and sentence vector j . To transform the values into cosine similarities, we divide element-wise by the outer product of the norms. This yields the cosine similarity between each pair of sentences:

$$SS_{i,j} = \frac{\mathbf{TF}_i \circ \mathbf{TF}_j^T}{\|\mathbf{TF}_i\| \otimes \|\mathbf{TF}_j\|} \quad (17)$$

The diagonal elements of the similarity matrix will always be 1, as the cosine similarity between a sentence and itself is always 1. We also clip the norm for the same reason mentioned in the KL Sum explanation, to prevent division by zero.

- 2) **Compute the sentence scores:** After constructing the similarity matrix, we sum the values across the rows or columns since the matrix is symmetrical. This step produces a vector that represents the sentence scores, indicating how connected each sentence is to all other sentences within the matrix.

$$\text{scores}_j = \sum_{i=1}^N SS_{i,j} \quad \text{for each } j = 1, 2, \dots, N \quad (18)$$

Finally, the sentences are ranked in descending order based on their sentence scores, and the top sentences are returned.

Using NumPy, we implemented the Graph Reduction summarization algorithm with the following code:

```

1  def GRSum(text: Union[str, List[str]], top_k:
    int = 3 ):
2      TF = preprocess(text)
3      # 1. Create a similarity matrix
4      col_norms = np.clip(np.linalg.norm(TF,
        axis=0), 1, None)
5      ss = np.dot(TF.T, TF) /
        np.outer(col_norms, col_norms)
6      # 2. Calculate the sentence scores
7      scores = np.sum(ss, axis=1)
8      # 4. Return the top_k sentences
9      return [text[int(idx)] for idx in
        np.argsort(scores)[:-1][:top_k]]

```

G. TextRank/LexRank

Both TextRank and LexRank are algorithms closely related to graph reduction, but with a few additional steps inspired by Google's PageRank algorithm. PageRank was originally developed by Google to rank the importance of websites based on their relationships to other websites[28], [29]. The core idea behind PageRank is that the importance of a website is determined by how many other websites link to it and how important those linking websites are, similar to graph reduction for websites, but adds an additional step of turning the adjacency matrix into a transitory matrix for a Markov chain and iterating the transitory matrix until it converges to a steady state. Below we implemented TextRank and LexRank using the TF matrix as an input:

- 1) **Transform the TF matrix into a similarity matrix.** Calculate the term overlap equation (19) or cosine similarity equation (17) between each sentence the same way we did in graph reduction. The choice between term overlap and cosine similarity differentiates between TextRank and LexRank, respectively:

$$SS_{i,j} = \frac{|\mathbf{TF}_i \cap \mathbf{TF}_j|}{\log(|\mathbf{TF}_i| + 1) + \log(|\mathbf{TF}_j| + 1)} \quad (19)$$

- 2) **Convert into a transitory matrix:** After constructing the similarity matrix, we need to convert it into a transitory matrix $\mathbf{J}$ where each element represents the probability of a sentence being represented by another sentence, which is calculated by their similarity score.

$$\text{colsum}(\mathbf{SS})_j = \sum_{i=1}^N SS_{i,j} \quad \text{for each } j = 1, 2, \dots, N \quad (20)$$

$$\mathbf{J}_{i,j} = \frac{SS_{i,j}}{\text{colsum}(\mathbf{SS})_j} \quad \text{for each } j = 1, 2, \dots, N \quad (21)$$

To ensure the Markov chain converges, the transition matrix must satisfy two key conditions: it must be irreducible (meaning no state is isolated, allowing transitions between all states) and aperiodic (meaning the chain does not return to a given state at fixed intervals). In the context of text data, the matrix may be irreducible but not aperiodic. This occurs when a row or column in the matrix represents a state where the only possible transition is to itself, effectively causing the Markov chain to get "stuck" in that state. This issue can arise, particularly due to aggressive preprocessing techniques that strip out meaningful words or leave sentences containing only specific terms that appear nowhere else in the text. In such cases, the matrix becomes reducible, with rows or columns that lack connectivity to other states.

To mitigate this, PageRank and TextRank/LexRank introduce a random jump mechanism, which allows for a small probability of transitioning to any state, regardless of the current position. This helps maintain a stochastic

transition matrix by incorporating a damping factor d , which is typically set to 0.85.

$$\mathbf{Z} = [d \cdot \mathbf{J} + (1 - d) \cdot \frac{1}{N}] \quad (22)$$

- 3) **Compute the sentence scores:** Set all of the initial values of the scores to an even distribution as shown in equation (23). Then, iterate the Markov chain in equation (24), until it converges to steady state i.e. until $\text{scores}_j^{k+1} - \text{scores}_j^k = 0$.

$$\text{scores}_j^0 = \frac{1}{N} \quad \text{for each } j = 1, 2, \dots, N \quad (23)$$

$$\text{scores}_j^{k+1} = \mathbf{M} \circ \text{scores}_j^k \quad (24)$$

The sentences are ranked in descending order based on their sentence scores, and the top sentences are returned.

Using NumPy, we implemented the TextRank and LexRank algorithm with the following code:

```

1  def TextRank(text: Union[str, List[str]],
2      top_k: int = 3, damping: float = 0.85,
3      max_iter: int = 100):
4      TF = preprocess(text)
5      # 1. Create a similarity matrix using Term
6      #      Overlap
7      log_sent_len = np.clip(np.log(np.sum(TF,
8          axis=0) + 1), 1e-10, None)
9      sim_matrix = np.dot(TF, TF.T) /
10         np.add.outer(log_sent_len,
11             log_sent_len)
12     # 2. Create Transition matrix
13     M = (1-damping) * (np.ones(ss.shape) /
14         ss.shape[0]) + damping * (ss /
15         np.clip(ss.sum(axis=1), 1e-10, None))
16     # 3. Perform power iteration until steady
17     #      state
18     scores = np.ones(ss.shape[0]) / ss.shape[0]
19     while max_iter > 0:
20         scores_next = np.dot(M, scores)
21         if np.allclose(scores, scores_next):
22             break
23         scores = scores_next
24         max_iter -= 1
25     # 4. Return the top_k sentences
26     return [text[int(idx)] for idx in
27         np.argsort(scores)[::-1][:top_k]]
28
29 def LexRank(text: Union[str, List[str]],
30     top_k: int = 3, damping: float = 0.85,
31     max_iter: int = 100):
32     TF = preprocess(text)
33     # 1A. Create a similarity matrix using
34     #      cosine sim
35     col_norms = np.clip(np.linalg.norm(TF,
36         axis=0), 1, None)
37     ss = np.dot(TF.T, TF) /
38         np.outer(col_norms, col_norms)
39     # 2. Create Transition matrix
40     M = (1-damping) * (np.ones(ss.shape) /
41         ss.shape[0]) + damping * (ss /
42         np.clip(ss.sum(axis=1), 1e-10, None))
43     # 3. Perform power iteration until steady
44     #      state
45     scores = np.ones(ss.shape[0]) / ss.shape[0]
46     while max_iter > 0:
47         scores_next = np.dot(M, scores)
48         if np.allclose(scores, scores_next):
49             break
50         scores = scores_next

```

```

33     max_iter -= 1
34     # 4. Return the top_k sentences
35     return [text[int(idx)] for idx in
36         np.argsort(scores)[::-1][:top_k]]

```

III. PERFORMANCE EVALUATION AND DISCUSSION

In this section, we will compare the performance of our methods, shown in the previous section, with *Sumy*'s implementation, the most popularly used statistical summarization package.

A. Dataset

To conduct a comprehensive performance comparison, we used the CNN/Daily Mail dataset, which is widely recognized and frequently used in the field of natural language processing. This dataset contains 312k of full-text articles and their corresponding summaries, making it ideal for evaluating various summarization models [21].

The summaries in the dataset can be classified into two categories: extractive and abstractive. Extractive summaries involve selecting specific sentences or phrases directly from the original text, while abstractive summaries generate new sentences that convey the same ideas and concepts but may not use identical wording. This distinction is crucial for our evaluation strategy.

When assessing the performance of our extractive model, it is most meaningful to compare its output against extractive summaries, specifically focusing on the text that a human annotator would likely highlight or extract for creating the summary. A high performance score would thus reflect the model's ability to identify and select the same key segments of text that a human would choose when creating a summary.

B. Scoring Method: ROUGE

The evaluation method used to generate scores between the candidate and true summaries is ROUGE, a widely used metric in automatic text summarization. ROUGE compares the overlap of n-grams between the candidate and reference summaries, assessing the quality of the generated summary.

1) ROUGE-1

ROUGE-1 measures the overlap of unigrams (single words) between the candidate and reference summaries. It calculates the proportion of unigram matches between the two summaries, reflecting the basic word-level overlap:

$$\text{ROUGE-1} = \frac{\sum_{w \in \text{unigrams}} \min(\text{count}(w, C), \text{count}(w, R))}{\sum_{w \in \text{unigrams in } R} \text{count}(w, R)} \quad (25)$$

Where w represents unigrams (individual words), C is the candidate summary, and R is the reference summary. This metric measures how well the candidate summary matches the individual words in the reference summary.

2) ROUGE-2

ROUGE-2 evaluates the overlap of bigrams (two consecutive words) between the candidate and reference

summaries. It captures the overlap of two-word sequences and is more sensitive to the structure and coherence of the text. This is done the same way as shown in equation (25) but w is every two consecutive words. This metric evaluates how well the candidate summary preserves the word pairings and syntactic structures present in the reference summary.

3) ROUGE-L

ROUGE-L measures the length of the longest common subsequence (LCS) between the candidate and reference summaries. The LCS identifies the longest sequence of words that appear in the same relative order in both summaries, without needing to be contiguous. This metric is particularly useful for evaluating the overall structural coherence of the summary:

$$\text{ROUGE-L} = \frac{\text{LCS}(C, R)}{|R|} \quad (26)$$

Where $\text{LCS}(C, R)$ represents the length of the longest common subsequence between the candidate summary C and the reference summary R , and $|R|$ is the total number of words in the reference summary. ROUGE-L rewards models that maintain the sequential structure and logical flow of ideas from the original text, even if exact word matches are not present.

4) Recall, Precision, and F1 Score

Additionally, ROUGE evaluation produces recall, precision, and F1 scores. These metrics are used to evaluate the trade-off between the coverage of the candidate summary and its accuracy in relation to the reference summary.

- a) **Recall** measures how many n-grams from the reference summary are captured by the candidate summary.
- b) **Precision** measures how many n-grams in the candidate summary are also present in the reference summary.
- c) **F1 Score** is the harmonic mean of precision and recall, providing a balanced measure of performance.

These additional metrics help provide a more comprehensive evaluation of the candidate summary's quality in relation to the reference.

C. Performance Comparison

Leveraging NumPy for matrix operations and NLTK for word tokenization and stemming, we re-implemented all of the discussed extractive summarization algorithms in fewer than 125 lines of code. As illustrated in Figure 3, our implementations consistently outperformed *Sumy* across all evaluated metrics. Both our methods and those from *Sumy* were tested on the same dataset of 1,000 CNN news articles, ensuring a fair and controlled comparison.

Our implementation of Luhn, SumBasic, KL Summarization, LSA, Graph Reduction, TextRank and LexRank all preformed similarly to each other. Conversely, *Sumy*'s performance between algorithms fluctuated, as shown in figure 3. Furthermore, the average performance gap, for each algorithm,

between our implementations and *Sumy*'s was approximately 26.7%.

The ROUGE-N and F1 scores are only one aspect of performance. In the next section, we turn our attention to what we consider the most compelling advantage of our approach: its substantial gains in computational efficiency.

Luhn				SumBasic			
F1 Score	Rouge-1	Rouge-2	Rouge-L	F1 Score	Rouge-1	Rouge-2	Rouge-L
Benchmark	.2350	.0720	.2190	Benchmark	.2210	.0590	.2040
Sumy	.2295	.0690	.2120	Sumy	.2035	.0488	.1884
KL Sum				LSA Sum			
F1 Score	Rouge-1	Rouge-2	Rouge-L	F1 Score	Rouge-1	Rouge-2	Rouge-L
Benchmark	.2330	.0700	.215	Benchmark	.2330	.0690	.2150
Sumy	.1909	.0504	.1714	Sumy	.2029	.0393	.1892
Graph Reduction				TextRank			
F1 Score	Rouge-1	Rouge-2	Rouge-L	F1 Score	Rouge-1	Rouge-2	Rouge-L
Benchmark	.2420	.0750	.2260	Benchmark	.2390	.0740	.2220
Sumy	.2060	.0550	.1890	Sumy	.2050	.0540	.1880
LexRank				--			
F1 Score	Rouge-1	Rouge-2	Rouge-L				
Benchmark	.2440	.0750	.2270				
Sumy	.2340	.0690	.2160				

Fig. 3. CNN/Daily Mail Rouge-N and F1 Score Comparison

D. Scalability Comparison

Luhn			SumBasic		
Speed (s)	1k Token Input	64K Token Input	Speed (s)	1k Token Input	64K Token Input
Benchmark	.0119	.2809	Benchmark	.0117	.2008
Sumy	.0310	.5824	Sumy	.0244	3.917
KL Sum			LSA Sum		
Speed (s)	1k Token Input	64K Token Input	Speed (s)	1k Token Input	64K Token Input
Benchmark	.0122	.8777	Benchmark	.0170	2.1254
Sumy	.1280	79.2006	Sumy	.0483	15.3919
Graph Reduction			TextRank		
Speed (s)	1k Token Input	64K Token Input	Speed (s)	1k Token Input	64K Token Input
Benchmark	.0118	.3824	Benchmark	.0126	.3876
Sumy	.0412	34.6298	Sumy	.0426	55.6597
LexRank			--		
Speed (s)	1k Token Input	64K Token Input			
Benchmark	.0171	.3863			
Sumy	.0550	92.240			

Fig. 4. Summary Speed (s) Comparison (1K vs 64K Token Input)

To generate the results shown in Figure 4, we measured the time required to produce summaries for two datasets: the CNN/Daily Mail, with an average of 1,000 tokens per article, and a book repository [22], averaging 64,000 tokens per book. Our methods, which operate with a time complexity of $O(N)$, demonstrated substantial speed advantages over *Sumy*, achieving a 12.7-fold increase in processing speed for 1K token inputs and an impressive 101.5-fold increase for 64K token inputs. This stark contrast highlights the scalability of the efficiency of NumPy's matrix-based computations and our novel adaptations of the equations, particularly in the multi-document summarization algorithms like KL-Sum and SumBasic.

IV. CONCLUSION

Although extractive text summarization has regained attention, much of the field still depends on *Sumy* as a baseline due to the lack of available implementations in many foundational

works. This dependence limits reproducibility and obscures the true capabilities of classical methods. To address this, we present a library that faithfully re-implements core extractive summarization algorithms as originally intended, while introducing modern computational optimizations and modular design.

Our results show that these implementations significantly outperform *Sumy*, achieving a **19.43% improvement in Rouge-N scores** and speedups of **12.7x and 101.5x** for 1K- and 64K-token inputs, respectively. This demonstrates that the shortcomings often associated with Bag-of-Words-based summarization arise from outdated implementations rather than the methods themselves.

By providing a transparent, efficient, and extensible baseline, this work enables fair comparison, reproducibility, and continued use of BoW techniques in modern NLP systems, including hybrid retrieval and summarization pipelines within RAG.

REFERENCES

- [1] Supriyono, Aji Prasetya Wibawa, et al. "A Survey of Text Summarization: Techniques, Evaluation and Challenges." **Natural Language Processing Journal**, vol. 7, June 2024, article no. 100070.
- [2] "Sumy - Module for Automatic Summarization of Text Documents and HTML Pages." *Sumy*, n.d., <https://miso-belica.github.io/textit{Sumy}/>.
- [3] Salihu, Shakirat Aderonke, et al. *Automatic Summarization of Legal Documents*. Department of Computer Science, University of Ilorin, Nigeria, 2023.
- [4] Wolyn, S., and S. J. Simske. "Summarization Assessment Methodology for Multiple Corpora Using Queries and Classification for Functional Evaluation." *Integrated Computer-Aided Engineering*, vol. 29, no. 3, 2022, pp. 227–239. <https://doi.org/10.3233/ICA-220680>.
- [5] Gupta, S., A. Sharaff, and N. K. Nagwani. "Biomedical Text Summarization: A Graph-Based Ranking Approach." *Applied Information Processing Systems*, Springer, 2022.
- [6] Verma, Pradeepika, and Hari Om. "Extraction Based Text Summarization Methods on User's Review Data: A Comparative Study." *Smart Trends in Information Technology and Computer Communications*, Springer, 2016, pp. 437–446. https://doi.org/10.1007/978-981-10-3433-6_42.
- [7] Shukla, Abhay, et al. "Legal Case Document Summarization: Extractive and Abstractive Methods and Their Evaluation." *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2022, pp. 1048–1064.
- [8] Meng, Rui, et al. "Bringing Structure into Summaries: A Faceted Summarization Dataset for Long Scientific Documents." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021, pp. 1080–1089.
- [9] Gusev, I. "Dataset for Automatic Summarization of Russian News." *Artificial Intelligence and Natural Language. AINL 2020*, Springer, 2020, pp. 129–139. https://doi.org/10.1007/978-3-030-59082-6_9.
- [10] Lamsiyah, S., et al. "Unsupervised Extractive Multi-Document Summarization Method Based on Transfer Learning from BERT Multi-Task Fine-Tuning." *Journal of Information Science*, vol. 49, no. 1, 2021, pp. 164–182. <https://doi.org/10.1177/0165551521990616>.
- [11] Chen, Qian, et al. "Distraction-Based Neural Networks for Document Summarization." University of Science and Technology of China, 2017.
- [12] Sah, S., et al. "Semantic Text Summarization of Long Videos." *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2017, pp. 989–997. <https://doi.org/10.1109/WACV.2017.115>.
- [13] de Vargas Feijó, D., and V. P. Moreira. "RulingBR: A Summarization Dataset for Legal Texts." *Computational Processing of the Portuguese Language. PROPOR 2018*, Springer, 2018, pp. 270–280. https://doi.org/10.1007/978-3-319-99722-3_26.
- [14] Barua, R., et al. "F-NAD: An Application for Fake News Article Detection Using Machine Learning Techniques." *2019 IEEE Bombay Section Signature Conference (IBSSC)*, 2019, pp. 1–6. <https://doi.org/10.1109/IBSSC47189.2019.8973059>.
- [15] Bhattacharya, Paheli, et al. "Incorporating Domain Knowledge for Extractive Summarization of Legal Case Documents." *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Law*, 2021.
- [16] Vimalaksha, A., et al. "Automated Summarization of Lecture Videos." *2018 IEEE Tenth International Conference on Technology for Education (T4E)*, 2018, pp. 126–129. <https://doi.org/10.1109/T4E.2018.00034>.
- [17] Yoo, Taewon, et al. "LectYS: A System for Summarizing Lecture Videos on YouTube." *Companion Proceedings of the 26th International Conference on Intelligent User Interfaces*, ACM, 2021, pp. 90–92. <https://doi.org/10.1145/3397482.3450722>.
- [18] Parahonco, Alexandr, and Mircea Petic. "Different Aspects on Extractive Text Summarization." *Workshop on Intelligent Information Systems (WIIS2023)*, 2023.
- [19] "Evaluation of Python Text Summarization Libraries." *International Journal of Engineering Development and Research*, vol. 9, no. 1, Jan. 2021, pp. 159–164. <https://rjwave.org/IJEDR/papers/IJEDR2101019.pdf>.
- [20] Sharma, Tripti, et al. "Evaluation of Python Text Summarization Libraries." Maharaja Surajmal Institute of Technology, 2021.
- [21] CNN_DailyMail Dataset. "cnn_dailymail." TensorFlow Datasets. https://www.tensorflow.org/datasets/catalog/cnn_dailymail
- [22] Ismael. (n.d.). IsmaelMousa/books · datasets at hugging face. IsmaelMousa/books · Datasets at Hugging Face. <https://huggingface.co/datasets/IsmaelMousa/books>
- [23] Giarelis, N., C. Mastrokostas, and N. Karacapilidis. "Abstractive vs. Extractive Summarization: An Experimental Review." *Applied Sciences*, vol. 13, no. 13, 2023, p. 7620. <https://doi.org/10.3390/app13137620>.
- [24] Luhn, Hans Peter. "The Automatic Creation of Literature Abstracts." **IBM Journal of Research and Development**, vol. 2, no. 2, 1958, pp. 159–165.
- [25] Nenkova, Ani, and Lucy Vanderwende. "The Impact of Frequency on Summarization." Microsoft Research, MSR-TR-2005-101.
- [26] Aria, H., and Lucy V. "Exploring Content Models for Multi-Document Summarization." **NAACL '09**, 2009, pp. 362–370.
- [27] Steinberger, Josef, and Karel Jezek. "Using Latent Semantic Analysis in Text Summarization and Summary Evaluation." **Proc. Isim**, 2004, pp. 93–100.
- [28] Mihalcea, Rada, and Paul Tarau. "TextRank: Bringing Order into Text." **Proc. of the 2004 EMNLP Conference**, 2004.
- [29] Erkan, Günes, and Dragomir R. Radev. "LexRank: Graph-Based Lexical Centrality as Salience in Text Summarization." **Journal of Artificial Intelligence Research**, vol. 22, 2004, pp. 457–479.